

Police Shootings Analysis

Brief

Working document for Salman Adnan

Concise companion to the Deep Dive

This is the concise companion to the project's Deep Dive document: same facts, higher altitude, key diagrams only. A glossary at the end covers terms this document uses without re-defining them inline. Nothing here is invented; every claim traces to the actual code or the project's README.

Contents

1	What it is	2
2	Architecture at a glance	2
3	<code>analyse_deaths.py</code>: the static-chart script	2
4	<code>app.py</code>: the interactive dashboard	3
5	Real bugs fixed while building this	4
6	Dependencies	4
7	Setup and usage	4
8	Licensing	4
9	Data and framing notes	5
10	Known limitations	5
11	Glossary	5

1 What it is

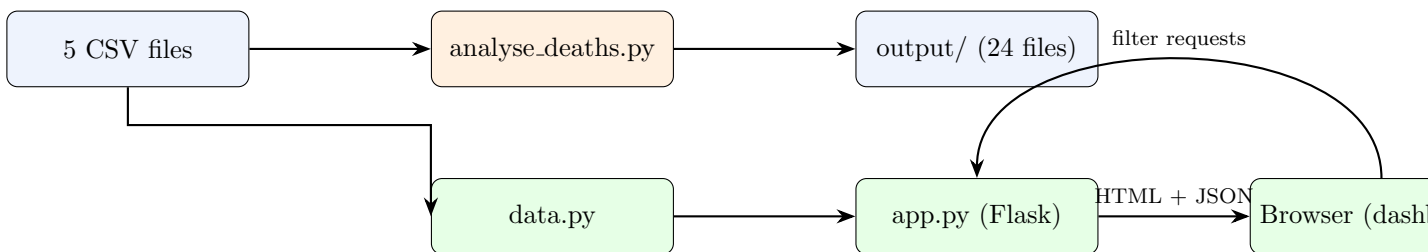
A data-analysis project built on *The Washington Post*’s fatal police shootings dataset: a fixed historical extract of **2,535 records**, 2015 through part of 2017, cross-referenced against four US Census datasets (household income, high-school completion rate, poverty rate, and race share, all by city). It ships two ways to view the results:

- **analyse_deaths.py**: a one-shot procedural script (the original exercise) that loads all five CSVs, cleans them, and writes 24 charts to **output/**, comparing matplotlib, seaborn, and plotly on the same data.
- **app.py**: a Flask-based interactive dashboard, added on top of the fatality data only, with a clickable US choropleth and cross-filtering by state, year, and armed status.

Origin: a guided “100 Days of Code” exercise, meant to practice joining inconsistent real-world datasets and comparing three plotting libraries’ APIs side by side.

Framing. This dataset records real deaths. Every chart here is descriptive (counts and correlations), never causal, and small cross-tabulated subsets (e.g. race within one city) can be a handful of records. This document keeps the README’s own neutral framing throughout.

2 Architecture at a glance



Two independent paths share nothing but the raw CSVs: the static script loads and cleans all five files inline (untouched since the dashboard was added); **data.py** is a small new module that loads and cleans only the fatality CSV, using the exact same steps, so the dashboard’s numbers can never drift from what the static script would compute on the same rows. The dashboard deliberately excludes the four Census CSVs: joining their city names to states isn’t unique enough to filter on cleanly, and state/year/armed status are the only fields with a natural drill-down structure in the fatality data.

3 analyse_deaths.py: the static-chart script

A ~390-line procedural script (no classes, one small helper), run top to bottom. It forces matplotlib’s headless **Agg** backend so it can run without a display, then builds 24 charts in sequence, saving each via a numbered **next_chart_path()** counter: matplotlib/seaborn charts as PNGs (**plt.savefig**), plotly charts as standalone HTML files (**fig.write_html**, needed to preserve interactivity). Coverage: poverty rate and high-school graduation rate by state (each in three libraries), their relationship (line/KDE/regression charts), racial makeup by state, gender/age/race breakdowns of people killed, armed status, signs of mental illness, top-10 cities by killings (with a race breakdown), two US choropleth maps, and killings per year.

Only 18 of the 24 charts (the PNGs) are committed to the repo; the 6 plotly HTML files are left out because each embeds a full copy of plotly.js and inflates to roughly 5 MB, ~30 MB total, and regenerate locally in under a minute.

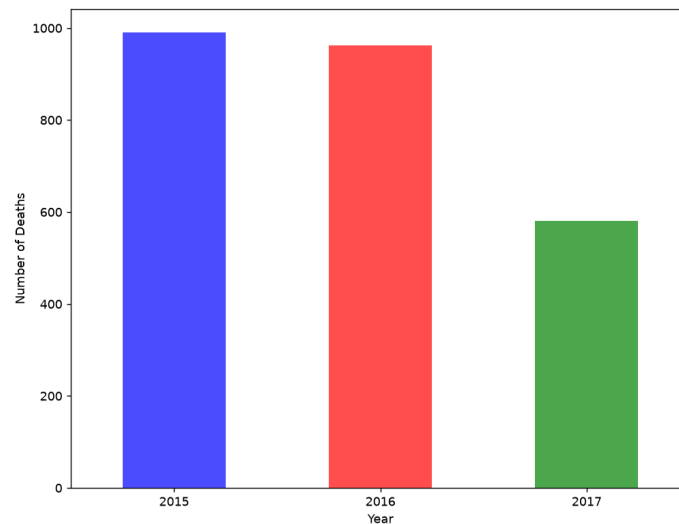
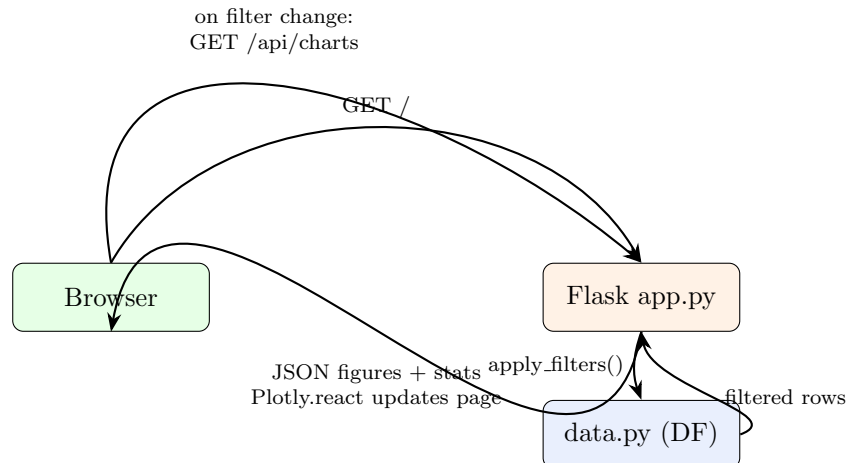


Figure 1: Actual output: `chart_24.png`, killings per year (991 / 2015, 962 / 2016, 582 in a partial 2017).

4 app.py: the interactive dashboard

Flask app, two routes: `/` renders the full page (with an initial, unfiltered payload embedded server-side to avoid an extra round trip), and `/api/charts` returns JSON for the current filter selection. The fatality data is loaded once at startup into an in-memory DataFrame and only ever filtered, never re-read from disk per request.



For each request, three `plotly.graph_objects` figures are rebuilt from the filtered rows: a US choropleth (with the selected state's border highlighted), a year bar chart, and a top-10-cities bar chart, plus a stat row (record count, percent with signs of mental illness, most-affected state, most common armed status) computed from those same filtered rows, so the numbers can never disagree with what the charts show. Every Plotly figure is serialized via `fig.to_json()` before reaching Flask's `jsonify()`, because Plotly figures carry numpy number types that Flask's own JSON encoder cannot handle directly.

On the frontend (`templates/index.html`, a single Jinja2 template with inline CSS/JS, light/-

dark theme via `prefers-color-scheme`): dropdown changes and map clicks both call the same `fetchAndRender()` function, which fetches `/api/charts` and calls `Plotly.react` (an in-place chart update, as opposed to a full teardown-and-rebuild) on all three charts, keeping hover/zoom/click state intact across filter changes.

5 Real bugs fixed while building this

- **Headless plotting.** `.show()` calls (written for an interactive notebook) don't work in a plain script with no display; fixed with the `Agg` backend and `savefig/write_html` instead.
- **Pandas version drift.** `value_counts().reset_index()` no longer always names the new column `'index'`; hardcoded lookups on that name broke in two places and were fixed to use the actual column names.
- **pycountry returns None for US state codes** (they aren't ISO country codes); fixed with a fallback to the original code.
- **Mixed date formats** (M/D/YYYY and DD/MM/YY rows in the same column) broke a plain `pd.to_datetime` call; fixed with `format='mixed', errors='coerce'`.
- **scipy is an unimported, lazily-loaded dependency** of pandas/seaborn's KDE plotting; added explicitly to `requirements.txt`.
- **Plotly figures aren't directly jsonify-able** because of embedded numpy types; fixed by routing every figure through Plotly's own `fig.to_json()` first.

6 Dependencies

pandas (loading/cleaning/aggregation), matplotlib and seaborn (two of the three compared static-plotting libraries), plotly (the third, and the engine behind every dashboard chart), pycountry (state/country code lookups for one choropleth), scipy (KDE math, used indirectly), Flask (the dashboard's web framework).

7 Setup and usage

```
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python app.py # dashboard: http://127.0.0.1:5000/
python analyse_deaths.py # static script: writes output/ (24 files)
```

No environment variables or external services needed; both entry points only read the CSV files already in the project folder.

8 Licensing

LICENSE is the **PolyForm Strict License 1.0.0**: permits use for any noncommercial purpose, personal research/study/hobby use, and use by charitable, educational, public-research, public-safety/health, environmental, or government institutions, but explicitly does *not* permit redistributing the software or creating modified/derivative works from it, and grants no right to sublicense or transfer the license. (This repository previously used MIT; PolyForm Strict is the current, correct license.)

9 Data and framing notes

Every chart is descriptive, not causal; small cross-tabulated subsets can represent a handful of records. Verified from an actual run: 2,535 total records, 2,428 male / 107 female, 633 (about 25%) with signs of mental illness; top **armed** values gun (1,398), knife (373), vehicle (177), unarmed (171), undetermined (117); top states California (424), Texas (225), Florida (154), Arizona (118), Ohio (79); year counts 991 (2015), 962 (2016), 582 (partial 2017). This is a fixed historical snapshot with no mechanism to pull updated data.

10 Known limitations

`analyse_deaths.py` remains one large procedural script with no function decomposition; the dashboard’s filters don’t extend to the four Census CSVs (no clean city→state join available); `convert_country_code` is a mostly-unneeded leftover from a country-level choropleth template; there’s no schema/dtype validation at load time; and the dataset itself is static, not a live feed.

11 Glossary

Term	Meaning
DataFrame	Pandas’ table-like data structure (rows + named columns).
Headless	Running with no graphical display attached.
Choropleth	A map with regions shaded by a numeric value.
KDE	Kernel Density Estimate; a smoothed probability-distribution curve.
Route	A URL path a web server listens on.
JSON	A plain-text structured-data format used for API responses.
Jinja2 template	An HTML file with server-filled placeholders/loops.
Correlation vs. causation	Trending together, vs. one thing actually causing the other; this project only measures the former.